

```
In [ ]: knitr::opts_chunk$set(echo = TRUE, warning=FALSE, message=FALSE, root.dir = './')  
getwd()
```

Part 1: deterministic vs stochastic model

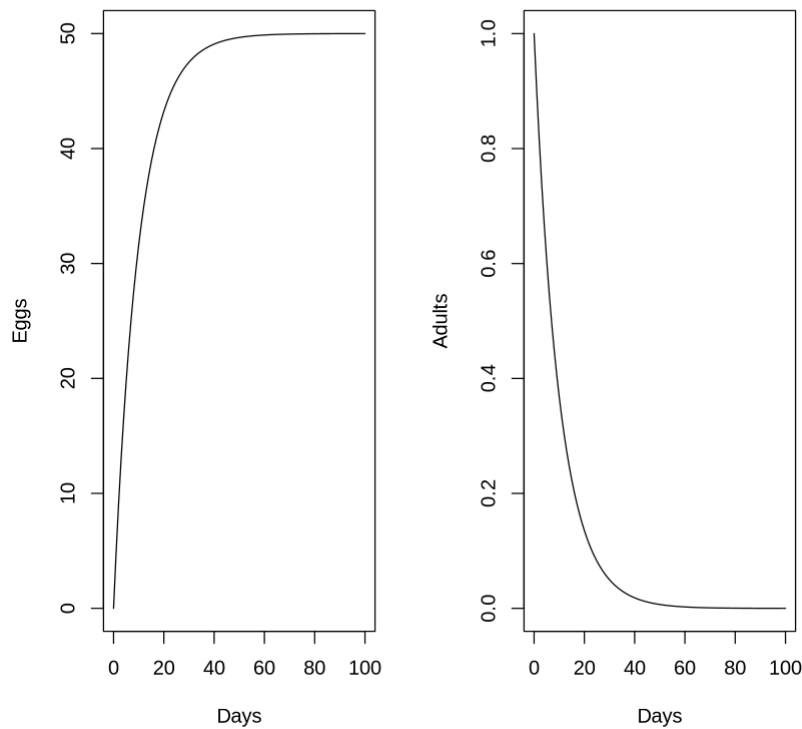
1. Deterministic model

Define the deterministic model

```
In [1]: library(deSolve)  
  
##### Introduction of a mosquito #####  
  
### Parameters  
params <- list(  
  fecundity = 5,  
  muA = 0.1 # mortality A  
)  
  
time <- 100  
state <- c(E = 0, A = 1)  
  
### ODE model with DeSolve  
population_model <- function(t, state, params) {  
  with(as.list(c(state, params)), {  
  
    dE <- fecundity * A  
    dA <- - muA * A  
    list(c(dE, dA))  
  })  
}
```

Run the deterministic model

```
In [2]: times <- seq(0, time, by = 1)  
det_out <- ode(y = state, times = times, func = population_model, parms = params)  
par(mfrow=c(1,2))  
  
plot(times, det_out[,2], type="l", xlab="Days", ylab="Eggs")  
plot(times, det_out[,3], type="l", xlab="Days", ylab="Adults")
```



2. Stochastic model

Define the stochastic model

```
In [3]: stochastic_model <- function(time, init, params) {
  out <- matrix(NA, nrow = time+1, ncol = 2)
  colnames(out) <- c("Eggs", "Adults")
  out[1, ] <- init

  for (t in 1:time) {
    E <- out[t, "Eggs"]
    A <- out[t, "Adults"]

    newE <- rpois(1, params$fecundity * A)

    dieA <- rbinom(1, A, params$muA)

    out[t+1, "Eggs"] <- E + newE
    out[t+1, "Adults"] <- A - dieA
  }
  return(out)
}
```

Run the stochastic model

```
In [4]: nrun <- 100
set.seed(123)

stoch_runs <- vector("list", length = nrun)

for (i in 1:nrun) {
  stoch_runs[[i]] <- stochastic_model(time, state, params)
}

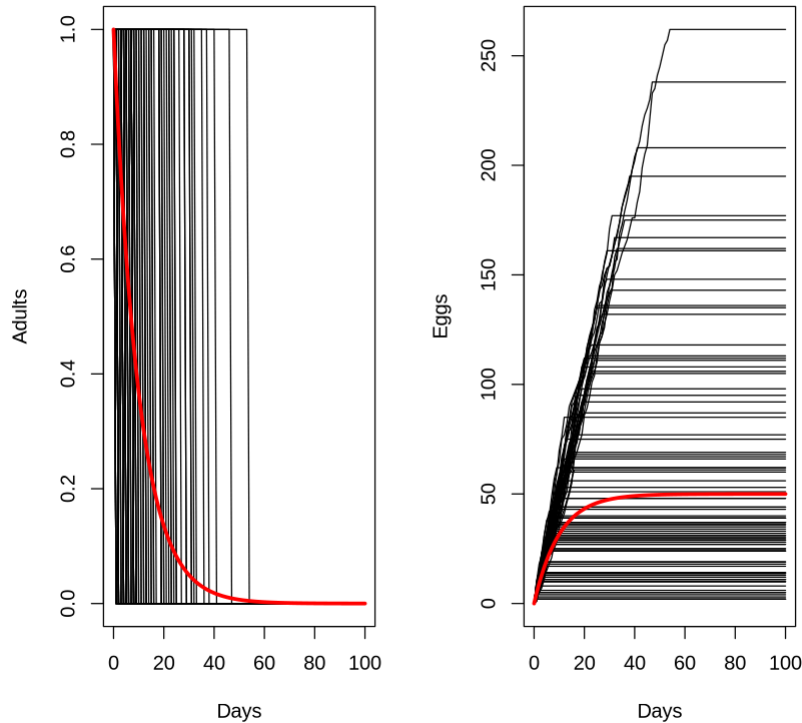
par(mfrow=c(1,2))
plot(times, det_out[,3], type="l", xlab="Days", ylab="Adults")
for(i in 1:length(stoch_runs))
```

```

lines(times,stoch_runs[[i]][,2])
lines(times,det_out[,3],col="red",lwd=3)

ymax=max(c(det_out[,2],sapply(stoch_runs, function(mat) max(mat[, 1]))))
plot(times,det_out[,2],type="l",xlab="Days",ylab="Eggs",ylim=c(0,ymax))
for(i in 1:length(stoch_runs))
  lines(times,stoch_runs[[i]][,1])
lines(times,det_out[,2],col="red",lwd=3)

```



```

In [5]: # number of simulations with few eggs
length(which(sapply(stoch_runs, function(mat) max(mat[, 1]))<=10))

```

12

3.SIR

```

In [6]: ## Parameters
N <- 10000      # total population
beta <- 0.3     # transmission rate
gamma <- 0.2    # recovery rate
time <- 180     # days

parms <- list(beta = beta, gamma = gamma, N = N)

## initial conditions: only 1 infectious
init <- c(S = N - 1, I = 1, R = 0)

sir_ode <- function(t, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <- beta * S * I / N - gamma * I
    dR <- gamma * I
    list(c(dS, dI, dR))
  })
}

times_det <- seq(0, time, by = 1)
det_out <- ode(y = init, times = times_det, func = sir_ode, parms = parms)
det_df <- as.data.frame(det_out)

```

```

stochastic_run <- function(time, init, params) {
  S <- I <- R <- numeric(time + 1)
  S[1] <- init["S"]; I[1] <- init["I"]; R[1] <- init["R"]

  for (t in 1:time) {
    St <- S[t]; It <- I[t]; Rt <- R[t]
    if (It <= 0) {
      S[t+1] <- St
      I[t+1] <- 0
      R[t+1] <- Rt
      next
    }

    p_inf <- 1 - exp(- params$beta * It / params$N)
    p_rec <- 1 - exp(- params$gamma)

    new_inf <- rbinom(1, St, prob = p_inf)
    new_rec <- rbinom(1, It, prob = p_rec)

    S[t+1] <- St - new_inf
    I[t+1] <- It + new_inf - new_rec
    R[t+1] <- Rt + new_rec

    S[t+1] <- max(0, round(S[t+1]))
    I[t+1] <- max(0, round(I[t+1]))
    R[t+1] <- max(0, round(R[t+1]))
  }
  cbind(S = S, I = I, R = R)
}

```

Run the SIR model

```

In [7]: nrun <- 100
        set.seed(123)

        stoch_runs <- vector("list", length = nrun)

        for (i in 1:nrun) {
          stoch_runs[[i]] <- stochastic_run(time, init, parms)
        }

```

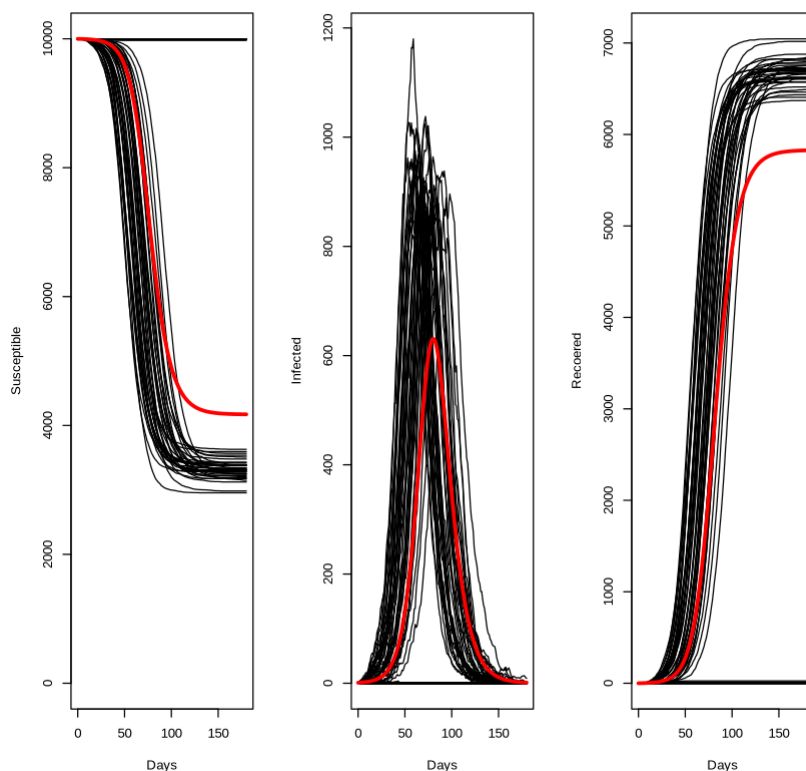
```

In [8]: par(mfrow=c(1,3))
        ymax=max(c(det_out[,2],sapply(stoch_runs, function(mat) max(mat[, 1]))))
        plot(times_det,det_out[,2],type="l",xlab="Days",ylab="Susceptible",ylim=c(0,ymax))
        for(i in 1:length(stoch_runs))
          lines(times_det,stoch_runs[[i]][,1])
        lines(times_det,det_out[,2],col="red",lwd=3)

        ymax=max(c(det_out[,3],sapply(stoch_runs, function(mat) max(mat[, 2]))))
        plot(times_det,det_out[,3],type="l",xlab="Days",ylab="Infected",ylim=c(0,ymax))
        for(i in 1:length(stoch_runs))
          lines(times_det,stoch_runs[[i]][,2])
        lines(times_det,det_out[,3],col="red",lwd=3)

        ymax=max(c(det_out[,4],sapply(stoch_runs, function(mat) max(mat[, 3]))))
        plot(times_det,det_out[,4],type="l",xlab="Days",ylab="Recoered",ylim=c(0,ymax))
        for(i in 1:length(stoch_runs))
          lines(times_det,stoch_runs[[i]][,3])
        lines(times_det,det_out[,4],col="red",lwd=3)

```



```
In [9]: # number of simulations with only 1 recovered (i.e. no further transmission)
length(which(sapply(stoch_runs, function(mat) max(mat[, 3]))==1))
```

32

Part 2: a short introduction to dynamAedes

This workflow uses the `dynamAedes` R package to simulate the introduction and population dynamics of the mosquito *Aedes aegypti* across Cyprus in 2024. The temperature data is extracted from the CERRA climate reanalysis and drives the biological rates in the model. Outputs include estimates of daily abundance and spatial spread.

1. Load Required Packages

```
In [ ]: # devtools::install_github("mattmar/dynamAedes@development")
library(terra)
library(tidyterra)
library(dynamAedes)
library(lubridate)
library(dplyr)
library(tidyr)
library(ggplot2)
```

2. Load and Preprocess Temperature Data

This section loads the daily 2m temperature raster data for 2024, reprojects it to its correct coordinate system (Lambert Conformal Conic), and crops it to the geographical extent of Cyprus to reduce computation time.

```
In [12]: myT <- rast("/srv/shared/t2m_CERRA_2024.nc")

crs_lcc <- "+proj=lcc +lat_1=50 +lat_2=50 +lat_0=50 +lon_0=8 +x_0=0 +y_0=0 +datum=WGS"
```

```

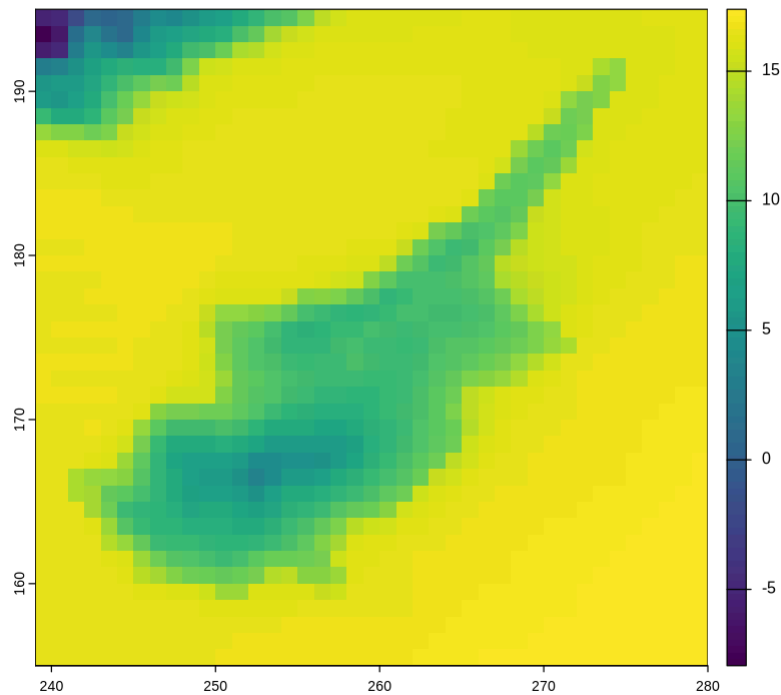
crs(myT) <- crs_lcc
names(myT) <- seq.Date(as.Date("2024-01-01"), by="day", length.out = nlyr(myT))

bbox <- ext(239, 280, 155, 195)
myT <- crop(myT, bbox)

myT <- flip(myT, direction = "vertical")
plot(myT[[1]])

```

Warning message:
 “[rast] unknown extent”



2.1 Apply a Sea Mask

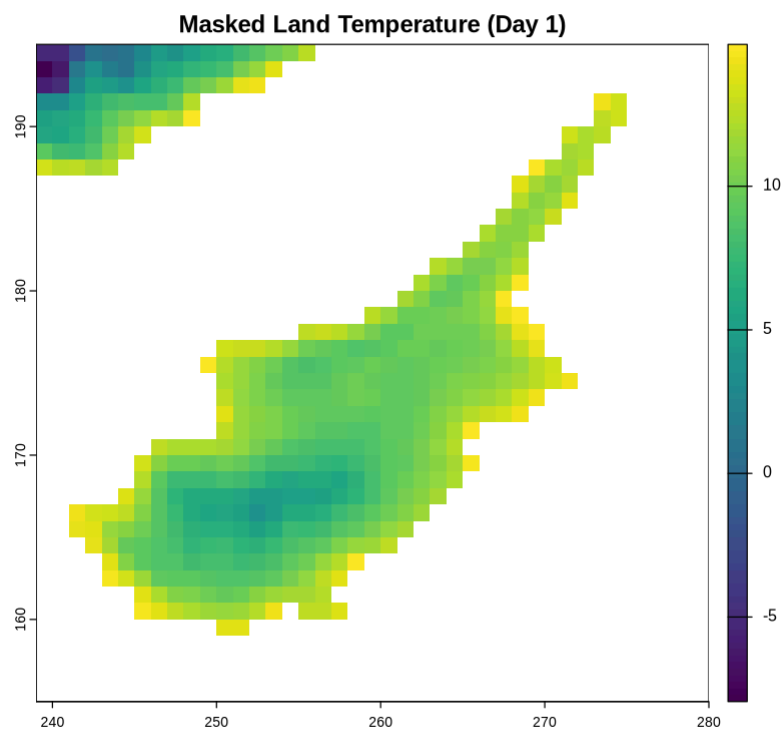
Temperatures over the sea are masked to simulate mosquito dynamics only on land.

```

In [13]: seaMask <- myT[[1]]
seaMask[seaMask > 15] <- NA
seaMask[!is.na(seaMask)] <- 1
myT <- myT * seaMask

plot(myT[[1]], main = "Masked Land Temperature (Day 1)")

```



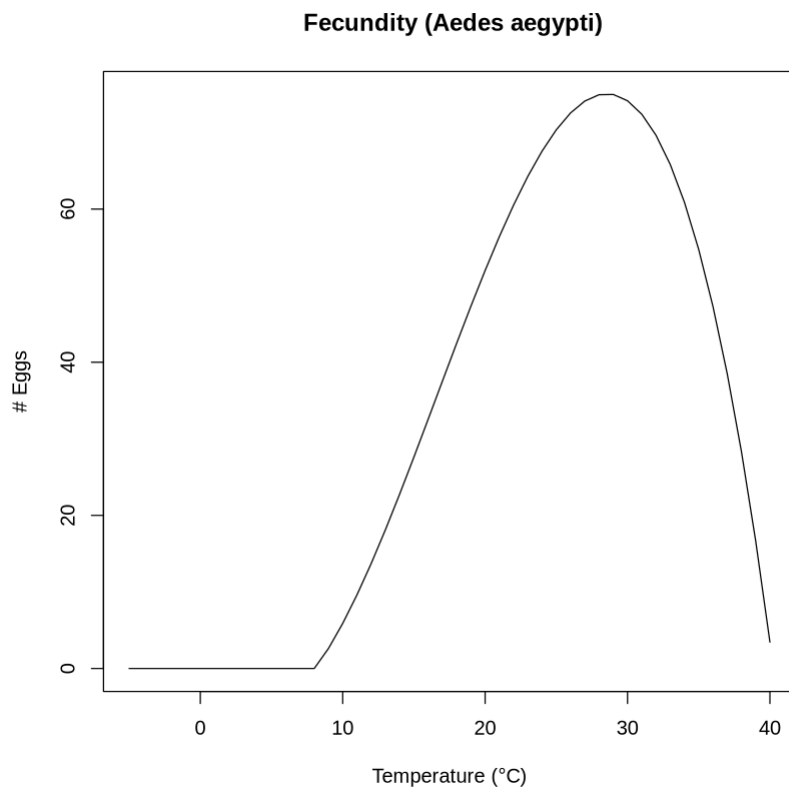
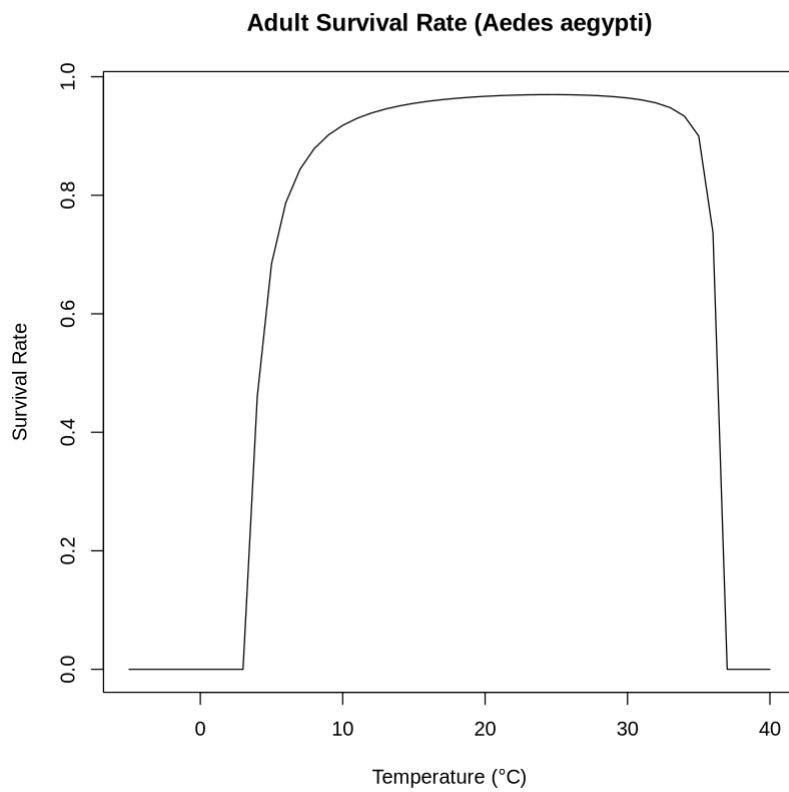
3. Explore Temperature-Dependent Life History Traits

Here we explore how temperature affects mosquito life-history parameters using `dynamAedes::get_rate`.

```
In [14]: ex.temp <- seq(-5, 40, by = 1)
my.r <- get_rate(temp = ex.temp, species = "aegypti", rate = "a.surv")
plot(ex.temp, my.r, type = "l",
     main = "Adult Survival Rate (Aedes aegypti)",
     xlab = "Temperature (°C)", ylab = "Survival Rate")

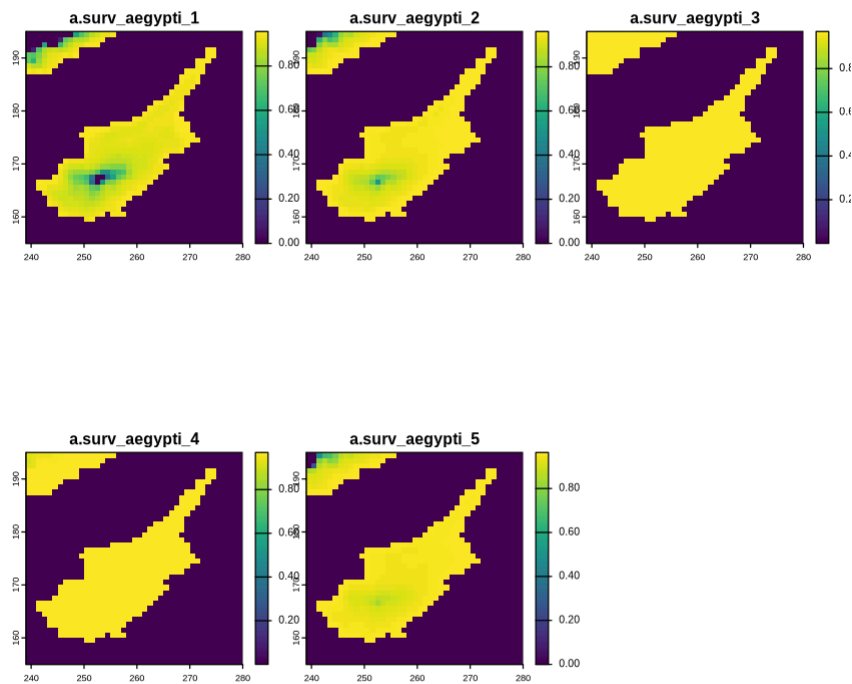
my.r <- get_rate(temp = ex.temp, species = "aegypti", rate = "a.ovl")

plot(ex.temp, my.r, type = "l",
     main = "Fecundity (Aedes aegypti)",
     xlab = "Temperature (°C)", ylab = "# Eggs")
```



Explore spatial variation in survival rates over a few selected days:

```
In [15]: #still dev version: add cores on tapp and fix the juvenile density
my.r <- get_rate(temp = myT[[c(10, 100, 200, 300, 350)]], species = "aegypti", rate =
plot(my.r)
```

4. Prepare Data for Simulation

Temperature data is converted to a format compatible with the `dynamAedes.m()` model: a matrix of temperatures and corresponding cell coordinates.

```
In [16]: df_temp <- as.data.frame(myT, xy = TRUE)
cc <- df_temp[, c("x", "y")]
w <- sapply(df_temp[, -c(1:2)], function(x) as.integer(x * 1000))
```

Define Simulation Parameters

```
In [17]: ## Define the day of introduction (January 1st is day 1)
str <- "2024-02-01"
## Define the end-day of life cycle (December 31st is the last day)
endr <- "2024-12-31"
## Define the number of eggs to be introduced
ie <- 10000
## Define the number of model iterations
it <- 10 # The higher the number of simulations the better
## Define the number of liters for the larval density-dependent mortality
habitat_liters <- 1000
## Define the number of parallel processes (for sequential iterations set nc=1)
cl <- 10
#subset w to get the first column of the matrix matching the day of introduction
w <- w[, as.numeric(format(as.Date(str), "%j")):ncol(w)]
```

5. Run the Aedes aegypti Model

```
In [ ]: simout <- dynamAedes.m(
  species = "aegypti",
  scale = "rg",
```

```

jhwv = habitat_liters,
temps.matrix = w,
cells.coords = as.matrix(cc),
startd = str,
endd = endr,
n.clusters = cl,
iter = it,
intro.eggs = ie,
compressed.output = FALSE,
seeding = TRUE,
verbose = FALSE
)

outname <- paste0("results/aegy_syms_uncompressed_workshopNicosia_it", it, "_", Sys.D
saveRDS(simout, outname)

```

6. Postprocessing

```

In [18]: simout <- readRDS("/srv/shared/aegy_syms_uncompressed_workshopNicosia_it10_2025-09-09
summary(simout)
dd <- max(simout)

```

Summary of dynamAedes simulations:

```

-----
Species:                Aedes aegypti
Scale:                  REGIONAL
Start Date:             2024-02-01
End Date:               2024-12-31
Number of Iterations:   10
Introduced Stage:       egg
Number Introduced:       10000
Is Output Compressed?:  No
Water in the System:    1000 L
Min days with population: 333
Max days with population: 333

```

6.1 PSI Maps

The PSI (Percentage of Successful Introductions) represents the proportion of simulations in which the mosquito population persisted in each cell.

```

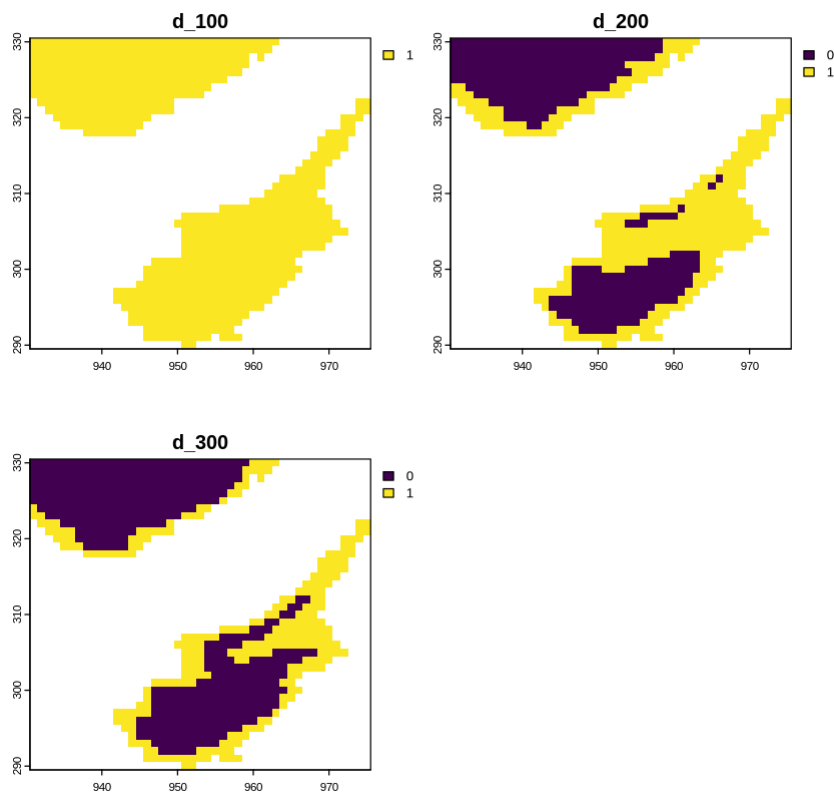
In [19]: psi.out <- psi_sp(input_sim = simout, eval_date = c(100, 200, 300), n.clusters = 2)
psi.out
plot(psi.out)

```

```

class      : SpatRaster
dimensions : 41, 45, 3  (nrow, ncol, nlyr)
resolution : 1, 1  (x, y)
extent     : 930.5, 975.5, 289.5, 330.5  (xmin, xmax, ymin, ymax)
coord. ref.:
source(s)  : memory
names      : d_100, d_200, d_300
min values :    1,    0,    0
max values :    1,    1,    1

```



6.2 Time Series of Abundance

```
In [20]: breaks <- c(0.25, 0.50, 0.75)
ed <- 1:dd

outdf <- rbind(
  adci(simout, eval_date = ed, breaks = breaks, stage = "Eggs", type = "0"),
  adci(simout, eval_date = ed, breaks = breaks, stage = "Juvenile", type = "0"),
  adci(simout, eval_date = ed, breaks = breaks, stage = "Adults", type = "0")
)

outdf$stage <- factor(outdf$stage, levels = c('Egg', 'Juvenile', 'Adult'))
outdf$Date <- rep(seq.Date(as.Date(str), as.Date(endr) - 2, by = "day"), 3)

ggplot(outdf, aes(x = Date, y = X50., group = stage, color = stage)) +
  ggtitle("Ae. aegypti Interquartile Range of Abundance") +
  geom_ribbon(aes(ymin = X25., ymax = X75., fill = stage), alpha = 0.2, color = NA) +
  geom_line(linewidth = 0.8) +
  labs(x = "Date", y = "Abundance", color = "Stage", fill = "Stage") +
  facet_wrap(~stage, scales = "free_y") +
  theme_light() +
  theme(legend.position = "bottom", text = element_text(size = 16))

#log10 transformed for cleared visualisation
ggplot(outdf, aes(x = Date, y = log10(X50.+1), group = stage, color = stage)) +
  ggtitle("Ae. aegypti Interquartile Range of Abundance") +
  geom_ribbon(aes(ymin = log10(X25.+1), ymax = log10(X75.+1), fill = stage), alpha =
  geom_line(linewidth = 0.8) +
  ylim(0, 10)+
  labs(x = "Date", y = "Abundance", color = "Stage", fill = "Stage") +
  facet_wrap(~stage, scales = "free_y") +
  theme_light() +
  theme(legend.position = "bottom", text = element_text(size = 16))
```

REGIONAL scale input simulation provided

Uncompressed simulation provided

Computing non-spatial Egg abundance...

REGIONAL scale input simulation provided

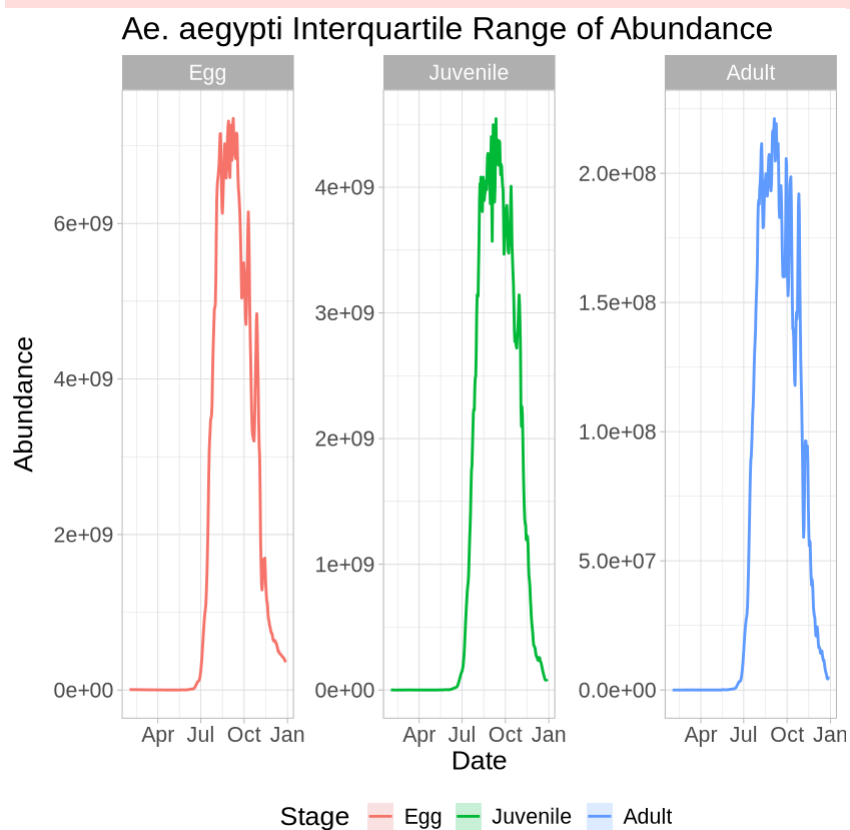
Uncompressed simulation provided

Computing non-spatial Juvenile abundance...

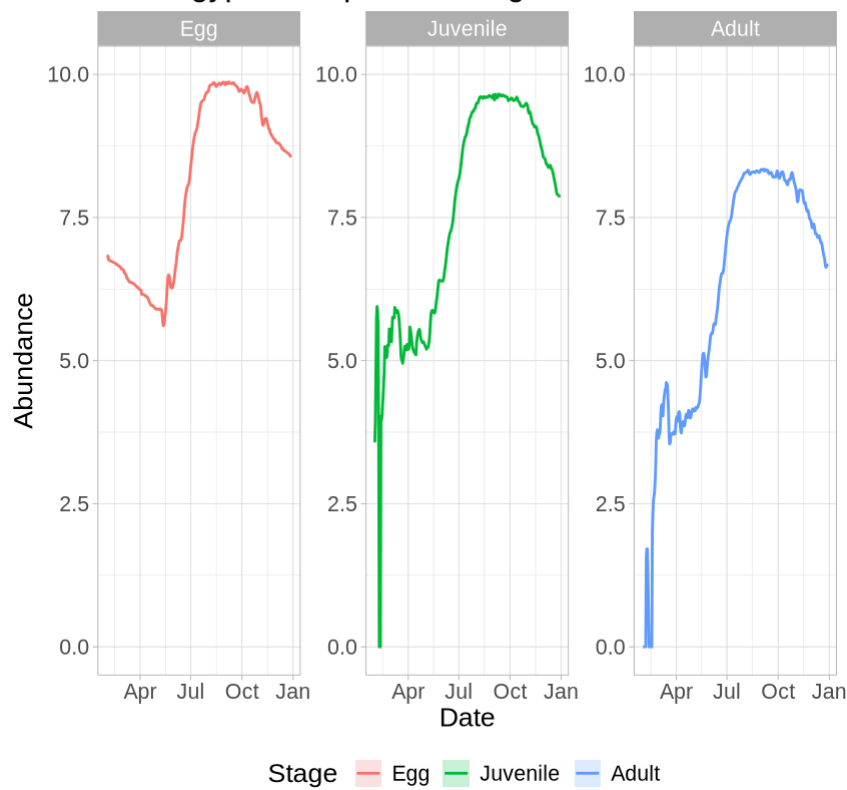
REGIONAL scale input simulation provided

Uncompressed simulation provided

Computing non-spatial Adult abundance...



Ae. aegypti Interquartile Range of Abundance



7. Spatial Summary of Female Adults

```
In [21]: breaks <- c(0.25, 0.50, 0.75)
ad.r <- adci(simout, eval_date = c(50, 100, 200, 300), breaks = breaks,
            stage = "Adults", type = "N", n.clusters = 10)
ad.r
ad.r <- ad.r$q_0.5

ggplot() +
  geom_spatraster(data = ad.r) +
  scale_fill_viridis_b(option = "viridis", direction = 1,
                      limits = c(0, 500000),
                      breaks = c(0, 50, 100, 1000, 10000, 25000, 50000),
                      oob = scales::squish, na.value = "transparent") +
  labs(x = "Longitude", y = "Latitude", fill = "Daily number of female adults") +
  facet_wrap(~lyr, ncol = 4) +
  theme_classic() +
  theme(legend.position = "bottom",
        text = element_text(size = 12),
        legend.text = element_text(size = 10),
        legend.key.size = unit(2, 'cm')) +
  coord_equal()
```

REGIONAL scale input simulation provided

Uncompressed simulation provided

Computing spatio-temporal abundance estimate at stage level.

```

$q_0.25
class       : SpatRaster
dimensions  : 41, 45, 4  (nrow, ncol, nlyr)
resolution  : 1, 1  (x, y)
extent      : 930.5, 975.5, 289.5, 330.5  (xmin, xmax, ymin, ymax)
coord. ref. :
source(s)   : memory
names       : day50, day100,   day200,   day300
min values  :    0,    0.0,    0.0,    0.0
max values  :   29,  155.5, 411189.2, 204821.8

```

```

$q_0.5
class       : SpatRaster
dimensions  : 41, 45, 4  (nrow, ncol, nlyr)
resolution  : 1, 1  (x, y)
extent      : 930.5, 975.5, 289.5, 330.5  (xmin, xmax, ymin, ymax)
coord. ref. :
source(s)   : memory
names       : day50, day100, day200,   day300
min values  :  0.0,   0.0,    2,    0.0
max values  : 33.5,  202.5, 423902, 204927.5

```

```

$q_0.75
class       : SpatRaster
dimensions  : 41, 45, 4  (nrow, ncol, nlyr)
resolution  : 1, 1  (x, y)
extent      : 930.5, 975.5, 289.5, 330.5  (xmin, xmax, ymin, ymax)
coord. ref. :
source(s)   : memory
names       : day50, day100,   day200,   day300
min values  : 0.00,   0.00,    4.0,    0.0
max values  : 40.75, 291.75, 436147.2, 205458.5

```

